



Python Programming - 2301CS404

Lab - 10

Roll No : 418

Name : Bhavik A. Parmar

01) WAP to read and display the contents of a text file. (also try to open the file in some other directory)

- in the form of a string
- line by line
- in the form of a list

```
In [21]: # 1. Read file as a STRING
fp = open("abc.txt", 'r')
data = fp.read()
print("----- File Content as String -----")
print(data)
fp.close()

# 2. Read file LINE BY LINE
fp = open("abc.txt", 'r')
print("\n----- File Content Line by Line -----")
for line in fp:
    print(line, end="")
fp.close()

# 3. Read file as LIST
fp = open("abc.txt", 'r')
data = fp.readlines()
print("\n\n----- File Content as List -----")
print(data)
fp.close()
```

```
# OR Only One Line
# fp = open("abc.txt", 'r')
# data = fp.readline()
# print(data)
# fp.close()

# OR
# fp = open("abc.txt", 'r')
# data1 = fp.readline()
# data2 = fp.readline()
# data3 = fp.readline()
# print(data1)
# print(data2)
# print(data3)
# fp.close()
```

----- File Content as String -----

Python uses open() to open a file for reading, writing, or appending.
 The read() method is used to read the entire content of a file.
 The write() method writes data into a file.
 The close() method closes the file and frees system resources.
 File modes like 'r', 'w', 'a', and 'x' specify how a file is accessed.
 The with statement automatically closes the file after use.
 readline() reads one line from a file at a time.
 input() is used to take user input from the keyboard.
 print() is used to display output on the screen.
 File I/O allows Python programs to permanently store data.

----- File Content Line by Line -----

Python uses open() to open a file for reading, writing, or appending.
 The read() method is used to read the entire content of a file.
 The write() method writes data into a file.
 The close() method closes the file and frees system resources.
 File modes like 'r', 'w', 'a', and 'x' specify how a file is accessed.
 The with statement automatically closes the file after use.
 readline() reads one line from a file at a time.
 input() is used to take user input from the keyboard.
 print() is used to display output on the screen.
 File I/O allows Python programs to permanently store data.

----- File Content as List -----

```
['Python uses open() to open a file for reading, writing, or appending.\n', 'The
read() method is used to read the entire content of a file.\n', 'The write() meth
od writes data into a file.\n', 'The close() method closes the file and frees sys
tem resources.\n', "File modes like 'r', 'w', 'a', and 'x' specify how a file is
accessed.\n", 'The with statement automatically closes the file after use.\n', 'r
eadline() reads one line from a file at a time.\n', 'input() is used to take user
input from the keyboard.\n', 'print() is used to display output on the scree
n.\n', 'File I/O allows Python programs to permanently store data.']
```

02) WAP to create file named "new.txt" only if it doesn't exist.

```
In [23]: # fp = open("new.txt", 'r')
# data = fp.read()
# print(data)
# fp.close()
# => Show Error (because it is no't exist.)
```

```
fp = open("new.txt", 'w')
s1 = "Hello World"
fp.write(s1)
fp.close()

# OR
try:
    fp = open("new.txt", "x")    # create only if not exists
    fp.write("Hello World")
    print("File created and data written.")
    fp.close()
except FileExistsError:
    print("File already exists.")
```

File created and data written.

03) WAP to read first 5 lines from the text file.

```
In [27]: fp = open("abc.txt", 'r')
data = fp.readlines()
res = data[:5]
print(res)
fp.close()

# fp = open("abc.txt", "r")
# count = 0
# while count < 5:
#     line = fp.readline()
#     if line == "":
#         break    # stop if file has less than 5 lines
#     print(line, end="")
#     count += 1
# fp.close()
```

['Python uses open() to open a file for reading, writing, or appending.\n', 'The read() method is used to read the entire content of a file.\n', 'The write() method writes data into a file.\n', 'The close() method closes the file and frees system resources.\n', "File modes like 'r', 'w', 'a', and 'x' specify how a file is accessed.\n"]

04) WAP to find the longest word(s) in a file.

```
In [29]: fp = open("abc.txt", "r")
data = fp.read()
words = data.split()
max_len = 0
longest_words = []

for w in words:
    if len(w) > max_len:
        max_len = len(w)
        longest_words = [w]
    elif len(w) == max_len:
        longest_words.append(w)

print("Longest word length:", max_len)
print("Longest word(s):", longest_words)
fp.close()
```

Longest word length: 13
Longest word(s): ['automatically']

05) WAP to count the no. of lines, words and characters in a given text file.

```
In [30]: fp = open("abc.txt", "r")
lines = fp.readlines()
line_count = len(lines)
word_count = 0
char_count = 0

for line in lines:
    words = line.split()
    word_count += len(words)
    char_count += len(line)

print("Number of Lines:", line_count)
print("Number of Words:", word_count)
print("Number of Characters:", char_count)
fp.close()
```

Number of Lines: 10
Number of Words: 104
Number of Characters: 582

06) WAP to copy the content of a file to the another file.

```
In [31]: fp1 = open("abc.txt", "r")
data = fp1.read()
fp2 = open("xyz.txt", "w")
fp2.write(data)
print("File content copied successfully.")
fp1.close()
fp2.close()
```

File content copied successfully.

07) WAP to find the size of the text file.

```
In [41]: fp = open("abc.txt", "r")
data = fp.read()
size = len(data)
print("File size:", size, "bytes")
fp.close()

# fp = open("abc.txt", 'rb')
# fp.seek(0, 2)
# file_size = fp.tell()
# fp.close()
# print(f"The size of the file {fp.name} is {file_size} bytes.")

# import os
# filename = "abc.txt"
# size = os.path.getsize(filename)
# print("File size:", size, "bytes")
```

File size: 582 bytes
The size of the file abc.txt is 591 bytes.
File size: 591 bytes

08) WAP to create an UDF named frequency to count occurrences of the specific word in a given text file.

```
In [43]: fp = open("abc.txt", "r")
search = input("Enter word to search: ")
count = 0
for line in fp:
    words = line.split()
    count += words.count(search)
print("Occurrence of", search, "=", count)
fp.close()
```

Occurrence of Python = 2

09) WAP to get the score of five subjects from the user, store them in a file. Fetch those marks and find the highest score.

```
In [45]: fp = open("marks.txt", "w")
for i in range(5):
    m = int(input("Enter marks: "))
    fp.write(str(m) + "\n")
fp.close()

fp = open("marks.txt", "r")
highest = 0
for line in fp:
    mark = int(line.strip())
    if mark > highest:
        highest = mark
print("Highest Score =", highest)
fp.close()
```

Highest Score = 30

10) WAP to write first 100 prime numbers to a file named primenumbers.txt

(Note: each number should be in new line)

```
In [46]: fp = open("primenumbers.txt", "w")
count = 0
num = 2
while count < 100:
    is_prime = True
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break
    if is_prime:
        fp.write(str(num) + "\n")
        count += 1
    num += 1
```

```
fp.close()
print("First 100 prime numbers written to primenumbers.txt")
```

First 100 prime numbers written to primenumbers.txt

11) WAP to merge two files and write it in a new file.

```
In [49]: fp1 = open("abc.txt", "r")
data1 = fp1.read()
fp1.close()

fp2 = open("xyz.txt", "r")
data2 = fp2.read()
fp2.close()

fp3 = open("merged.txt", "w")
fp3.write(data1)
fp3.write("\n")
fp3.write(data2)
fp3.close()
print("Files merged successfully into merged.txt")
```

Files merged successfully into merged.txt

12) WAP to replace word1 by word2 of a text file. Write the updated data to new file.

```
In [50]: fp = open("abc.txt", "r")
data = fp.read()
fp.close()

word1 = input("Enter word to replace: ")
word2 = input("Enter new word: ")

new_data = data.replace(word1, word2)

fp2 = open("updated.txt", "w")
fp2.write(new_data)
fp2.close()
print("Word replaced successfully. Updated data written to updated.txt")
```

Word replaced successfully. Updated data written to updated.txt

13) Demonstrate tell() and seek() for all the cases(seek from beginning-end-current position) taking a suitable example of your choice.

```
In [58]: fp = open("abc.txt", "rb") # binary mode

print("Initial position:", fp.tell())

# 1. From BEGINNING
fp.seek(6, 0)
print("\nAfter seek from beginning:", fp.tell())
print("Data:", fp.read(6).decode())

# 2. From CURRENT position
fp.seek(1, 1)
```

```
print("\nAfter seek from current:", fp.tell())
print("Data:", fp.read(6).decode())

# 3. From END
fp.seek(-5, 2)
print("\nAfter seek from end:", fp.tell())
print("Data:", fp.read().decode())

fp.close()
```

Initial position: 0

After seek from beginning: 6

Data: uses

After seek from current: 13

Data: pen()

After seek from end: 586

Data: data.

In []: